

Wolfram Language を使って、グラフの平面埋め込みを求めてみよう

[1] によると、Wolfram Language というものを使って、グラフの平面埋め込みを求めることができる。Wolfram Language は数学、機械学習といった科学的用途に特化したプログラミング言語で、普通の言語にはデフォルトで備わっていないような高度な機能が使用できる。この言語は、最初は Mathematica 内で使用するために開発されたが、その後さらに様々な機能が付け加えられ、独立して今の形になった。したがって、今回は Wolfram Language を用いてグラフの平面埋め込みができるか判定し、できるのであれば、その埋め込みの一例を求めている。

環境の準備

まず、Wolfram Language の実行環境を用意する。これには、公式サイトの Playground(<https://www.wolfram.com/language/#playground>)にアクセスする、Wolfram Cloud の Notebook(<https://www.wolframcloud.com/>)を使う、もしくは Wolfram Engine(<https://www.wolfram.com/engine/>)をダウンロードする必要がある。

Playground(必要最小限)

Playground を使用した場合は、コードをブロックに書いて、右端の Run を押せば実行できる。実行例を Figure 1 に示す。

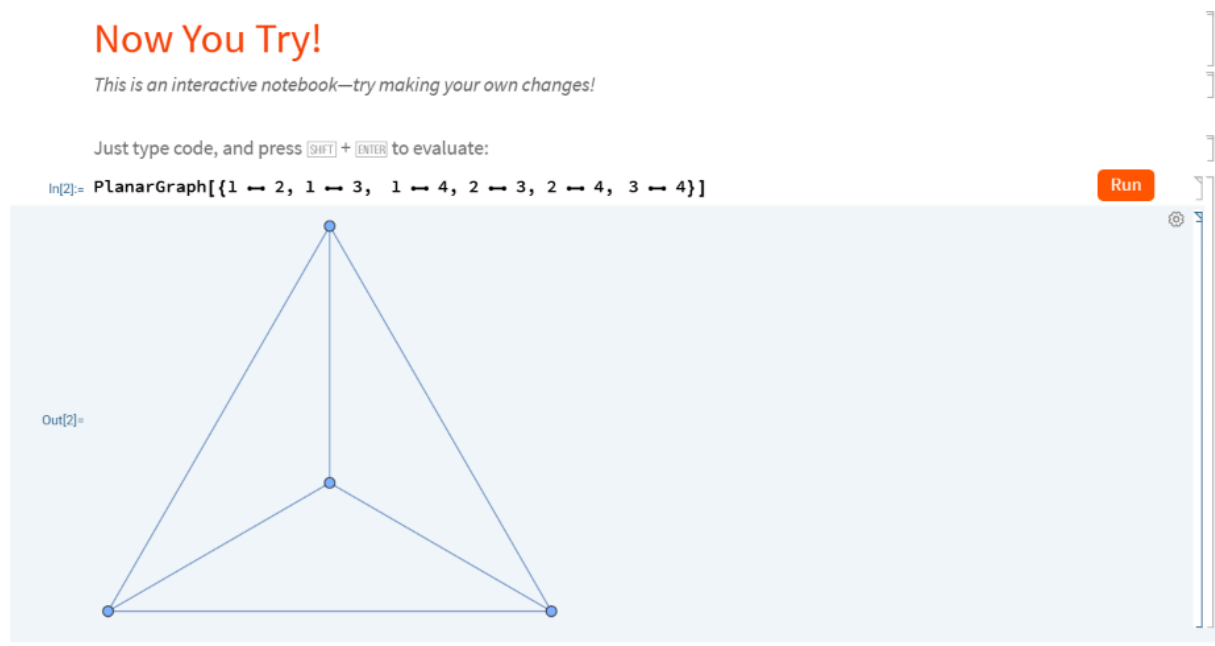


Figure 1: 実行例

Wolfram Cloud の Notebook(推奨)

(<https://www.wolframcloud.com/>)にアクセスし、ログインする（アカウントがなければ作成する）。すると次の Figure 2 のような画面に遷移する。

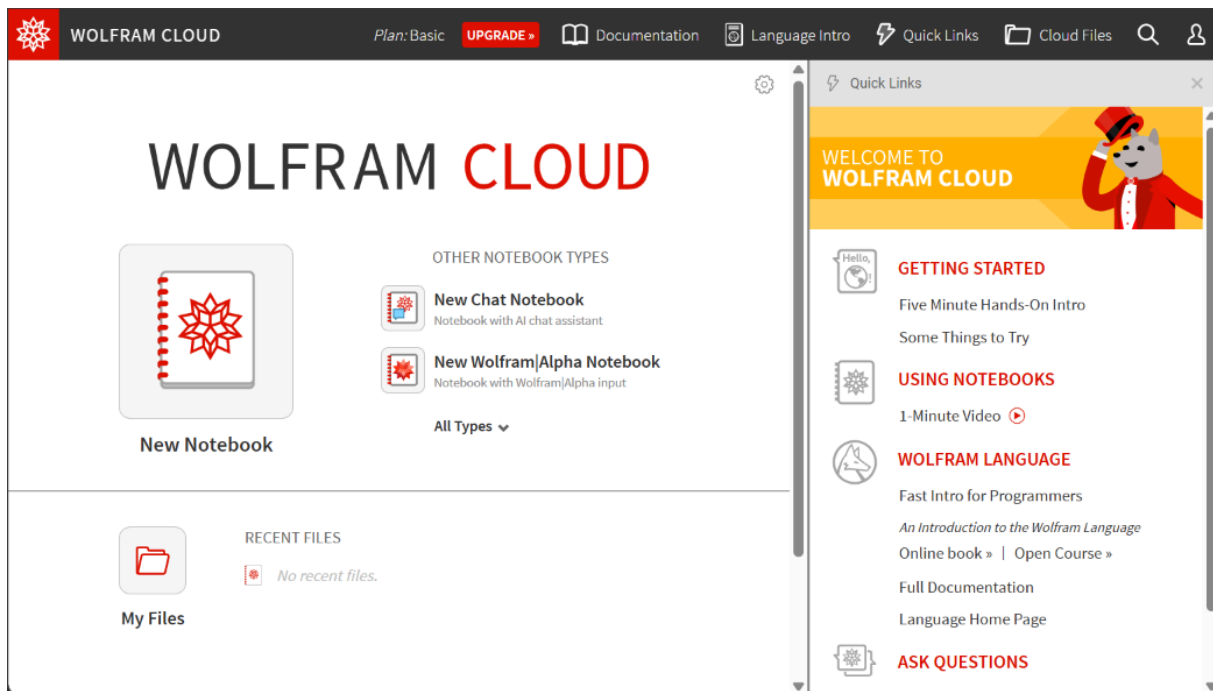


Figure 2: クラウドの画面

New Notebook を押し、左側の画面にコードを貼りつける。Shift+Enter(もしくは上の Evaluation) で実行する。

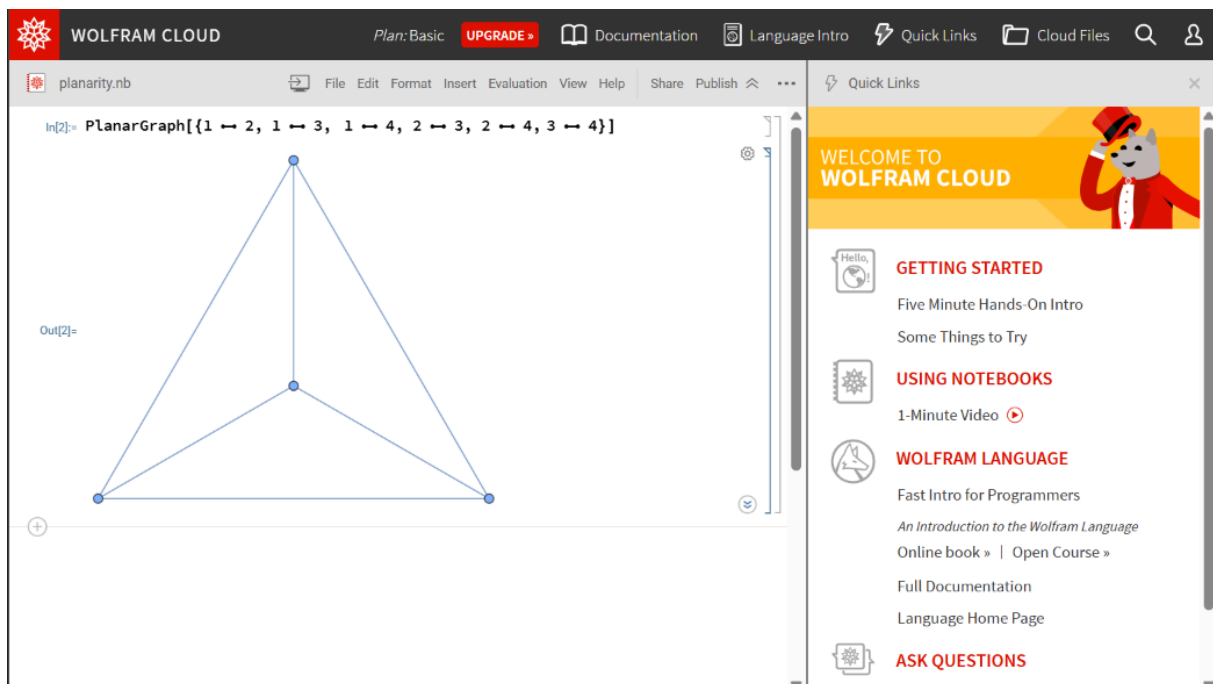


Figure 3: クラウドでの実行結果

ローカル環境(Wolfram Engine)(玄人向け)

Wolfram Engine を使用する場合は、[2] を参考にしながら進めると良い。ここでは、それに従いながら、[2] に書かれていないもう少し先(VS Code 用拡張機能の導入)までを説明する。

まず最初に、Wolfram Engine をダウンロードしなければならない。したがって、公式サイト(<https://www.wolfram.com/engine/>)から Wolfram Download Manager をダウンロードし起動する。ダウンロードボタンを押した後もまだ公式サイトは閉じないこと。(次のステップで

戻ってくる) 私は Windows 10(21H2)+用のバージョン 15.0 の Wolfram Download Manager をダウンロードした。(別のパッケージマネージャーを使う方法もある) そのダウンロードしたインストール用 exe ファイルを起動後、Wolfram Engine のダウンロードが始まるのだが、普通に 2.16GB 分のダウンロードを要求されるので、ディスク残量と次の予定の時間には注意しておくこと。

この間に、Wolfram Engine を使うのに必要なライセンス(License)を取得する。先ほどの公式サイトに戻ると、ページの内容が変化しており、次の Figure 4 のようなページになる。ページ中央の Get your license ボタンを押すと、別のページ(<https://account.wolfram.com/access/wolfram-engine/free>)に遷移する。Wolfram ID を作成するよう要求されるので作成する。(持っている場合はログインしておく)

するとこのページ(Figure 5)に遷移するはず(もし遷移しなければ、<https://account.wolfram.com/access/wolfram-engine/free> に手動で移動する)なので、利用規約に同意し Get license を押す。

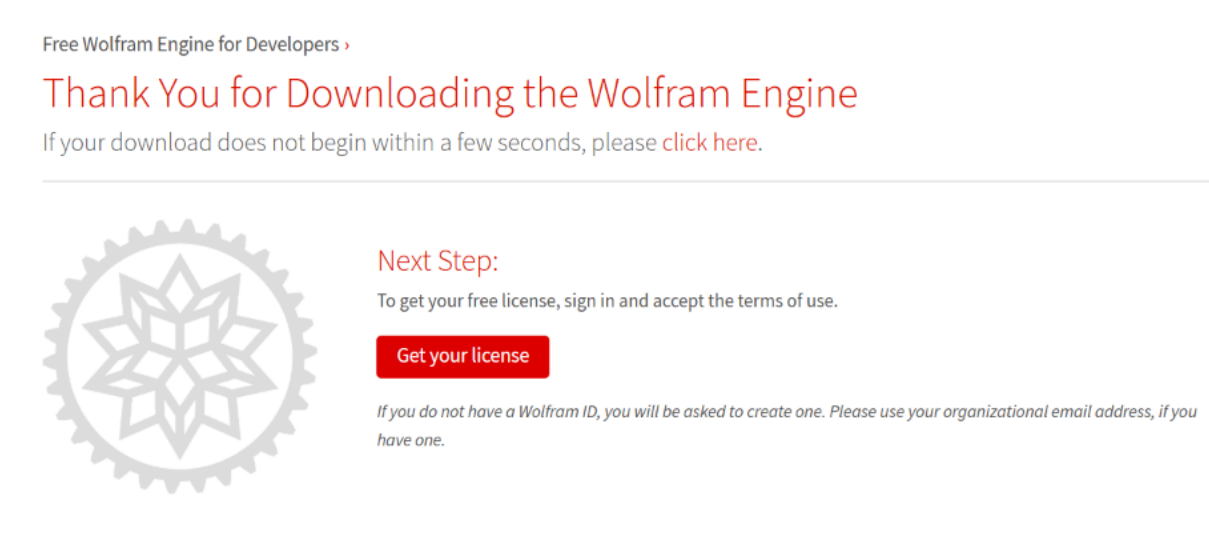


Figure 4: ダウンロードボタンを押した後のページ

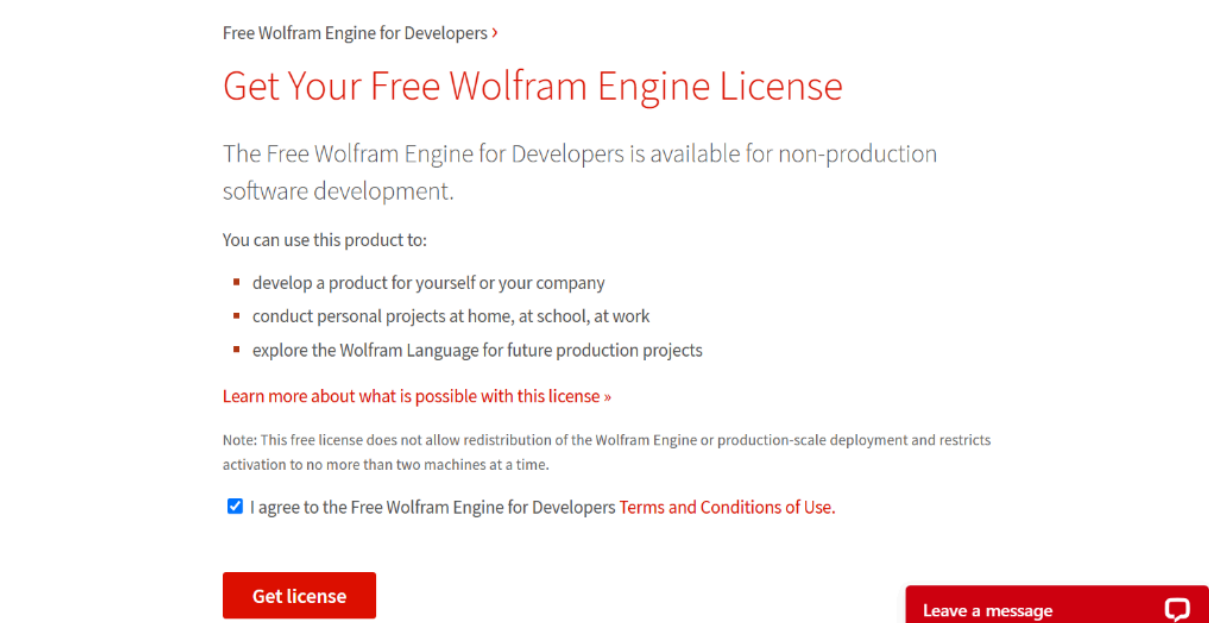


Figure 5: 利用規約同意画面

次に Wolfram Download Manager のウィンドウに戻り、(ダウンロードが終わるのを待って、) Wolfram Engine をウィンドウ内のボタンから起動しセットアップを実行する。今度はさらに 7GB くらい要求されるので、暇つぶしできるものを用意しておくこと。

ダウンロード後、ようやく Wolfram Engine が起動する。(起動しない場合は Wolfram Script を検索して手動で起動する) Figure 6 のような画面が開き、今度は Wolfram ID(email)とその password を要求されるので、頑張って入力する。

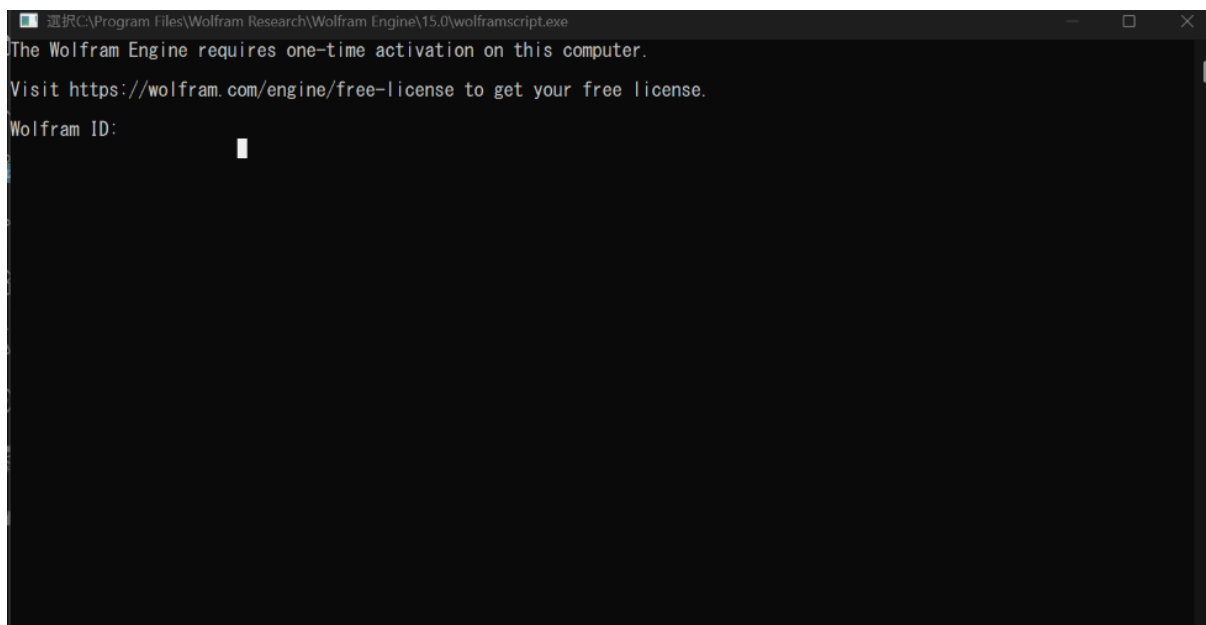


Figure 6: ID 入力画面

入力し終わると、ようやく Wolfram Engine が Activated になり、使えるようになる。¹ ここまでが [2] の内容である。

さて Wolfram Engine をどう使えばいいのかわだが、Playground や Notebook とは異なり、コマンドライン上でそのまま例をコピーしてもグラフを描画してくれない。したがってさらに外部ツールを用いる必要がある。今回は VSCode 上に Wolfram Language 拡張機能 (WolframResearch.wolfram, Figure 7) をインストールする。Figure 7 に示されているように、同じ名前の拡張機能が複数あるので、Wolfram Research が提供しているものを選ぶ。

¹このタイミングで、私の環境だとなぜか CTRL+C でペーストから、右クリックでペーストに突然なったので、注意してください。

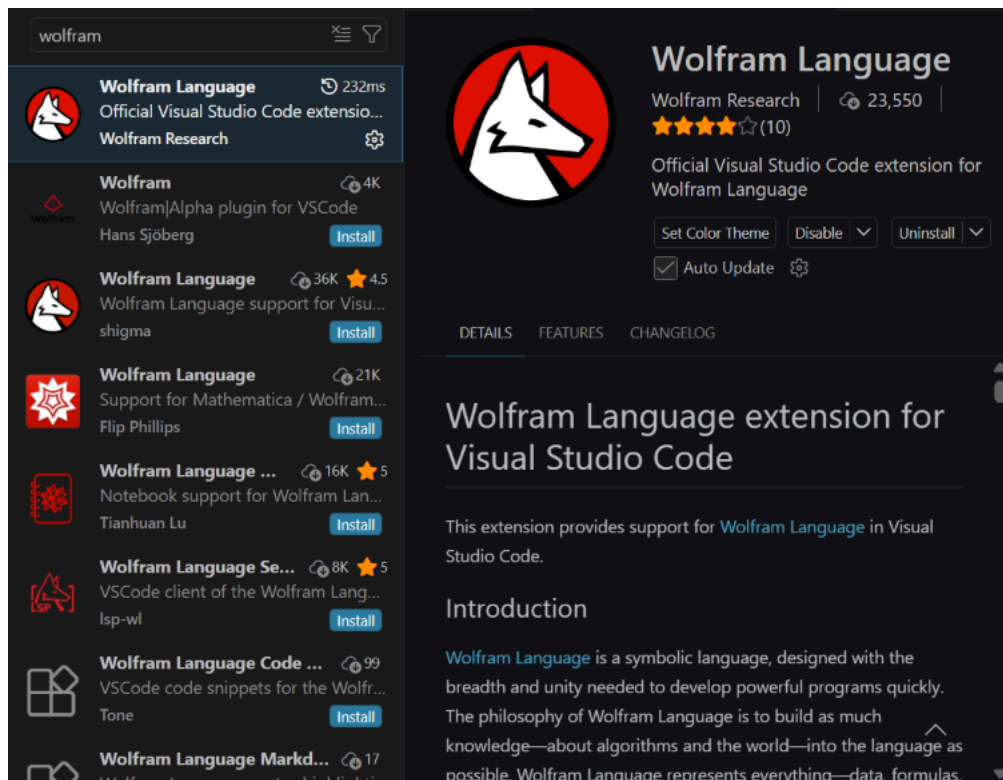


Figure 7: VSCode の Wolfram 用拡張機能

本来はこの拡張機能をインストールすれば自動的に Wolfram Engine の場所を見つけてくれるはずだが、私の環境では上手くいかずエラーが出た²ので、手動で設定する。

Wolfram: System Path を C:\Program Files\Wolfram Research\Wolfram Engine\15.0 に設定しなおす。そして、再起動すると VSCode 内で Wolfram Language を使えるようになる。

拡張機能が準備できたので、適当なファイル名でファイルを作り、拡張子を .vsnb にする。(planarity.vsnb とか) そして、そのファイルを開くと Figure 8 のように表示される。

²Kernel is not found in the default location. Either change "System Kernel" in the configuration or Download Wolfram Engine for kernel support.

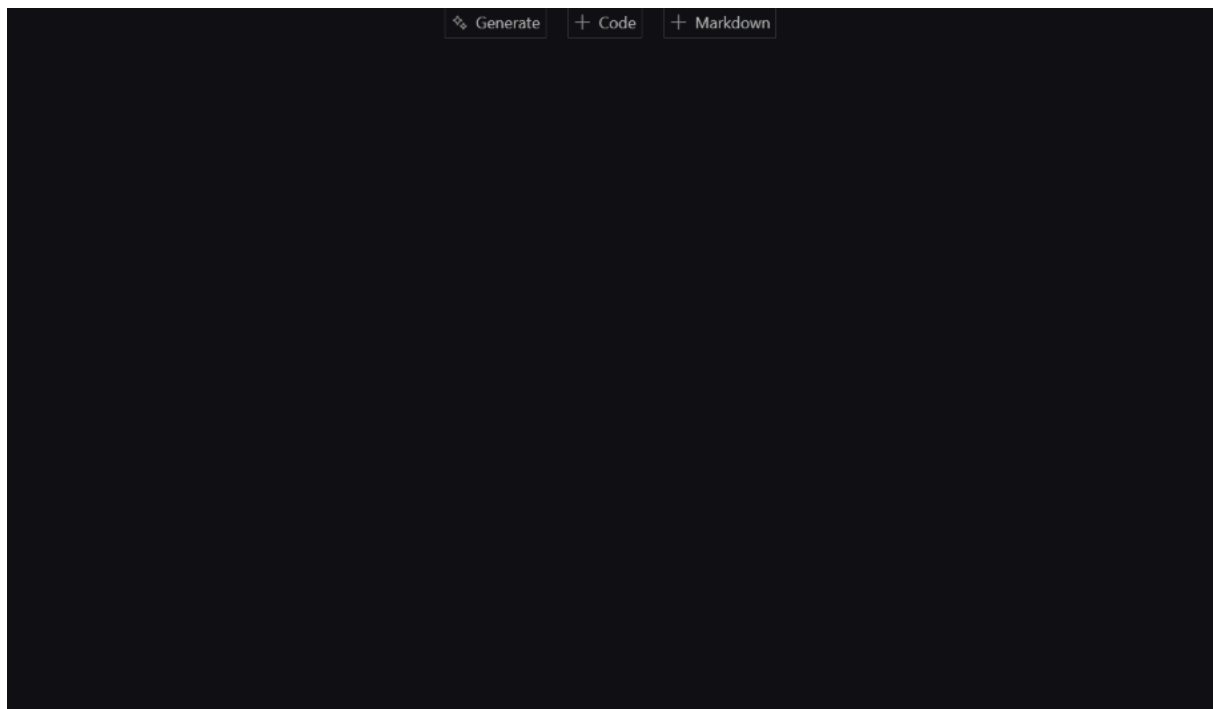


Figure 8: VSCode で開かれた.vsnb ファイル

上部中央の+Code を押す。そして、コードを貼って左端の三角形を押すと実行される。実行結果は Figure 9 のようになる。

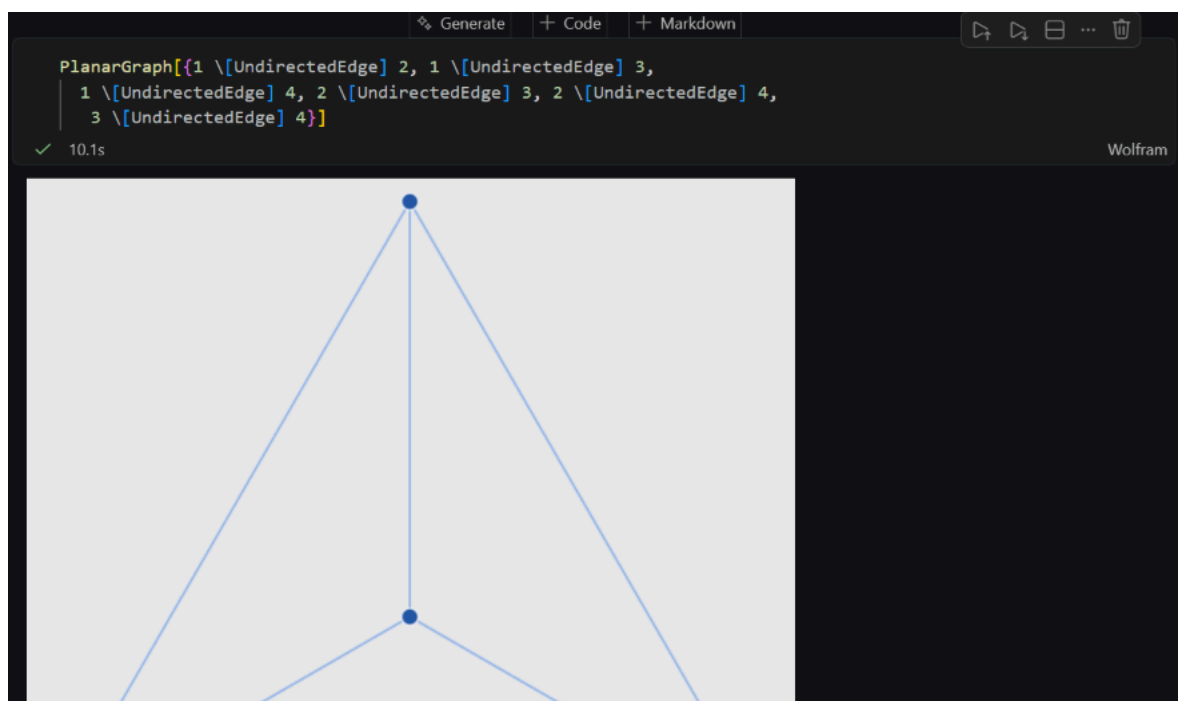


Figure 9: ローカル環境での実行結果

実際に平面埋め込みを書く

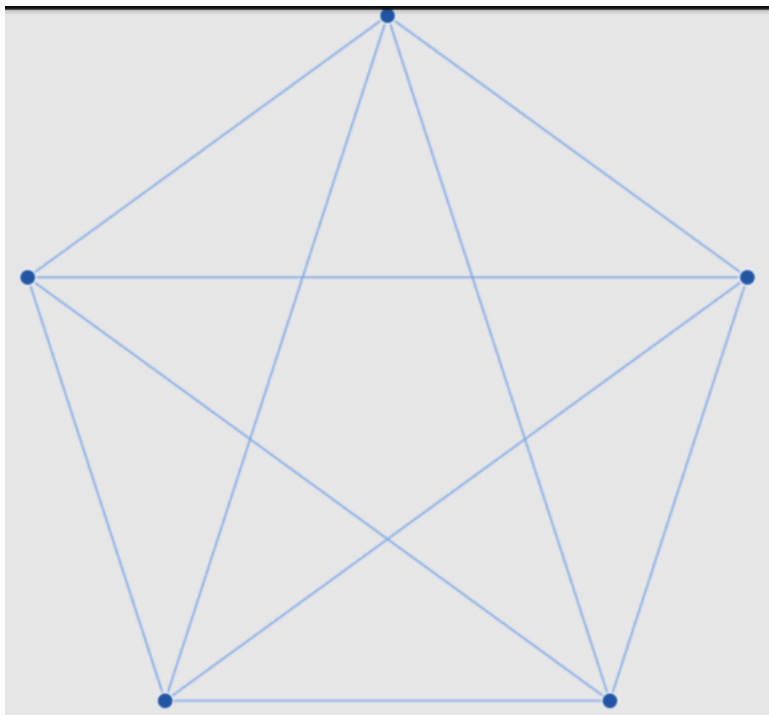
まずは、あえて平面埋め込みできないグラフ、 K_5 （頂点数 5 の完全グラフ）を平面埋め込みさせてみる。

このグラフは Wolfram Language 側のプリセットとして CompleteGraph[頂点数] という形ですでに用意されている。([3] を参照) というわけで実際に埋め込みを試みる。

実行コード:

```
g = CompleteGraph[5]
PlanarGraph[EdgeList[g]]
```

実行結果:



PlanarGraph::nplanar: Graph[{1 $\times$ 2, 1 $\times$ 3, 1 $\times$ 4, 1 $\times$ 5, 2 $\times$ 3, 2 $\times$ 4, 2 $\times$ 5, 3 $\times$ 4, 3 $\times$ 5, 4 $\times$ 5}] is not a planar graph.

ちょっと文字化けしているけれども、しっかりとエラーが出て平面埋め込みできなかった。最後に、適当なグラフの平面埋め込みを表示させる。結果は Figure 10 に示す。

実行コード:

```
{GridGraph[{2,3,2}], PlanarGraph[EdgeList[GridGraph[{2,3,2}]]]}
```

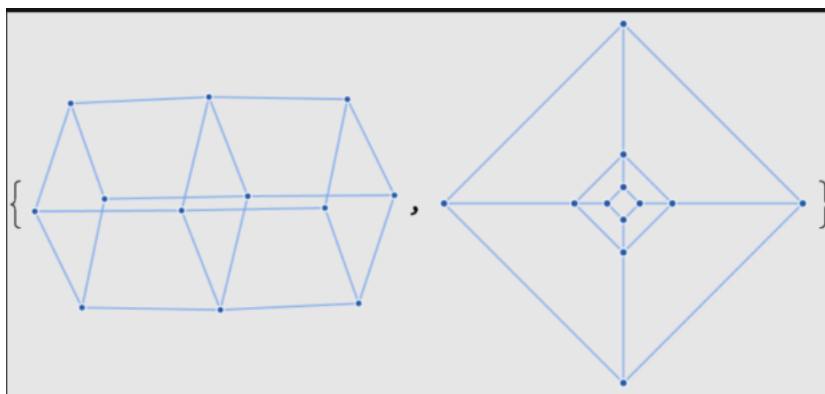


Figure 10: 適当なグラフの平面埋め込み（左のグラフを右で平面埋め込みしている）

Bibliography

- [1] “Planar Graph.” [Online]. Available at: <https://mathworld.wolfram.com/PlanarGraph.html>
- [2] “How do I set up the Wolfram Engine?.” [Online]. Available at: <https://support.wolfram.com/45743>
- [3] “CompleteGraph.” [Online]. Available at: <https://reference.wolfram.com/language/ref/CompleteGraph.html>

おまけ:Typst でブログを書きたい!!!!

Typst を私は LaTeX の代替として愛用している。しかし、はてなブログは LaTeX の一部分しかサポートしていないし、Typst がサポートしている他の書き出し形式は pdf,png と svg しかない。しかし、pdf にして公開するのは個人的にはあまり好きではない。というのも、pdf は紙への印刷用に最適化されているため、画面上で読むときに不便だからだ。特に、一度 pdf にしてしまうと、そのデータを再び取り出すのが非常に大変になるのが好ましくない。(普通の文章であればそれなりにコピペして再利用できるが、見出しや箇条書きといった文章構造が崩れてしまうし、そもそも、図表や数式はコピペできないことが多い。)というかそもそも、はてなブログに pdf を上げることができない。かといって画像データで出力してブログにするのはさすがに論外である。そうなると、困ってしまってわんわんわわん わんわんわわん。³

救世主現る。その名は *Typst 0.13*。

しかし、およそ一年半前、Typst 0.13 がリリースされた。それは黒船であった。pdf,png,svg がさながら三国志のように争いながら鎖国体制を築いていた、我らが島 Typst に、HTML という新しい国がやってきたのである。

黒船、いきなり沈む。

さてでは実際に HTML に書き出してみよう。適当なサンプルデータ (Listing 1) を用意したので、そのデータを実際に Typst 0.13 で PDF(Figure 11)と HTML(Figure 12)で書き出してみる。

³一応、出力される pdf に `pdf.attach("file-itself.typ")` のようにして元の Typst ファイルを添付しておくことはできる。しかしそもそも、pdf ファイルにさらに別のファイルが添付できることを知っている人がどれくらいいるだろうか。そして、そのファイルを開ける pdf ビューワーがどれくらいあるだろうか。少なくとも筆者は今回調べてみるまで知らなかった。


```

#import "@preview/cetz:0.5.0"
#set text(lang: "jp")
私は文章です。#footnote("我は脚注なり")

凄い数式:$f(x) = 1/x^2 \sin (1/x)$

#figure(caption: "I am a table", table(
  columns: 2,
  table.header("桁", "数"),
  "1", "3",
  "2", "1",
  "3", "4",
))

#figure(caption: "吾輩は図である", html.frame(cetz.canvas({
  import cetz.draw: *
  catmull((-0.90, 1.65), (-1, 0), (1, 0), (0.90, 1.65), (0, 2), close: true, name:
  "i")
  for-each-anchor("i", name => {
    circle("i." + name, radius: .1, fill: blue)
  })
})))

```

Listing 1: サンプルデータ

私は文章です。¹

凄い数式: $f(x) = \frac{1}{x^2} \sin(\frac{1}{x})$

桁	数
1	3
2	1
3	4

Table 1: I am a table

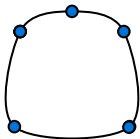


Figure 1: 吾輩は図である

¹我は脚注なり

Figure 11: PDF での出力結果

私は文章です。¹

凄い数式:

桁数

1 3

2 1

3 4

Table 1: I am a table

Figure 1: 吾輩は図である

¹我は脚注なり

Figure 12: HTML での出力結果

そう、この HTML 出力は実験的な機能なので、まだ数式や図が上手く出力できないのである。というわけで黒船は沈みましたとさ。ちゃんちゃん。

黒船、復活。

ところがどっこい。およそ半年前くらいにリリースされた 0.15 から実は HTML 出力が数式に対応した。ちゃんと画像形式ではなく MathML 形式で出力されるため、構文情報を失わずに HTML 化できる。え？じゃあなんで一回失敗したのかって？そんなの筆者が怠惰で Typst のバージョンを更新していなかったからに決まっているじゃあないか。何を言っとるんだね君は。しかし今の段階でも、図の出力はまだ上手くいかない。そこで、今回は図の部分だけ SVG でレンダリングするという回避策を用いてとりあえずなんとかした。⁴

⁴具体的には `html.frame(cetz.canvas(...))` とすることで、`cetz.canvas` の出力を SVG に変換して HTML に埋め込むことができる。なお、普通に `png` を読み込んでいる場合はこの回避策を用いる必要はなく、そのまま `image("path/to/image.png")` とすればよい。ただ、画像ファイルの中身を `image` タグに埋め込む形で出力されるので、人によっては好ましくない場合があるかもしれない。

私は文章です。¹

凄い数式: $f(x) = \frac{1}{x^2} \sin\left(\frac{1}{x}\right)$

桁数

1 3

2 1

3 4

Table 1: I am a table

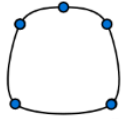


Figure 1: 吾輩は図である

¹私は脚注なり

Figure 13: HTML での出力結果その 2

できた！

というわけで、HTML 出力ができるようになったので、このままブログにして公開します。タイトルは「Wolfram Language で平面描画を求め、Typst で結果をブログにしてみよう！」にしておきましょうか！

できてない！

筆者ははてなブログを利用しているのですが、このブログサービスでは出力された HTML を直接使うといういろいろな問題が発生します。例えば、

1. そもそも HTML ではなく Markdown
2. head や html タグが入っていて、ブログの HTML 構造が崩れる。
3. 画像を埋め込むときに、img タグの src 属性に直接画像ファイル埋め込むと、文字数制限が何かでエラーがでて上手く表示できなくなる。

そもそも HTML ではなく Markdown

実は Markdown 内で HTML は使用できる。なのでとくに問題はない。

head や html タグが入っていて、ブログの HTML 構造が崩れる。

デフォルトの出力だと、html, head, body タグといった不必要なタグが出力されてしまう。まだ実験的機能なので、今の段階ではこれを消す方法は提供されていない。なので、Typst で書き出した HTML をそのままブログに貼り付けると、ブログの HTML 構造が崩れてしまう。なので、Typst で書き出した HTML の中身だけを取り出して、ブログに貼り付ける必要がある。今回は応急処置として、そのまま張り付けると背景が真っ赤になって、警告されるようにしておく。背景が赤いと、ブログの HTML 構造が崩れていることが一目でわかるので、Typst で書き出した HTML の中身と style タグだけを取り出して、ブログに貼り付ける必要があることを思い出せるはずである。たぶん。

画像を埋め込むときに、img タグの src 属性に直接画像ファイル埋め込むと、文字数制限か何かでエラーがでて上手く表示できなくなる。

img タグの src 属性に直接画像ファイルを埋め込むと、HTML が非常大きくなってしまいおそらく文字数制限でエラーがでて上手く表示できなくなる。なので、画像ファイルはブログのサーバーにアップロードして、その URL を img タグの src 属性に指定する必要がある。今回は応急処置として、img タグの src 属性に仮の URL を指定しておく。ブログに貼り付けるときは、img タグの src 属性をブログのサーバーにアップロードした画像の URL に書き換える必要がある。画像データを HTML とまとめて上げる方法がない以上、この手順は踏まざるを得ない。

これらの処置を自動的に施す Typst の関数を作ると以下ようになる。

```

#let show_blog_with_typst(body) = context {
  set text(lang: "jp")
  // ここにcontextを二重につけてはいけない。
  show image: it => {
    if target() == "html" {
      // TODO:Typst 1.50では、デフォルト動作では画像データ自体をHTMLのimgタグのsrc属性に埋め込む
      仕様になっている。
      // これを回避しているのだが、渡されたsourceをそのままimgタグに埋め込む以外の方法がないため、
      フォルダー内に別のtypstファイルがあって、それをincludeしている場合などに、相対パスの位置が変わり時々
      壊れる。
      // https://typst.app/docs/reference/foundations/path/ pathをstringに戻せば、img
      タグにstr(path(it.source))のようにして渡すことで相対パスの位置を正しく計算できるの可能性があるが、
      現状ではそれも不可能なので、仕方なくsourceをそのままimgタグに埋め込む仕様にしている。]
      if (it.source.ends-with(".pdf")) {
        // PDFを埋め込む場合はembedタグを使う。embedタグはPDFを埋め込むことができるが、imgタグで
        パスを指定してもPDFを埋め込むことはできない。
        html.elem("embed", attrs: (
          src: it.source,
          style: "max-width: 100%; height: auto;",
        ))
      } else {
        html.elem("img", attrs: (src: it.source, style: "max-width: 100%; height:
auto;"))
      }
    } else {
      it
    }
  }
  // TODO:html,headやbodyタグを消したいが、Typst 1.50では不可能そうなので応急処置として、そのま
  ま張り付けると背景が真っ赤になって、警告されるようにしておく。背景が赤いと、ブログのHTML構造が崩れてい
  ることが一目でわかるので、Typstで書き出したHTMLの中身とstyleタグだけを取り出して、ブログに貼り付ける
  必要があることを思い出せるはず。
  if target() == "html" {
    html.elem("div", attrs: (id: "takehere", style: "background: red;"))[
      #html.elem(
        "h1",
      )
    ]
    // [警告:このTypstで書き出したHTMLはdiv\#takehereの中身とstyleタグだけを取り出して、ブログ
    に貼り付ける必要があります。]
    #body
  }

  この記事はTypst 1.50の実験的なHTML出力機能を用いて生成されました。

  ソースコードのリポジトリは#link\("https://github.com/manybear-al/blog-typst", "こち
  ら"\)

  // 現時点では生成元のTypstファイル名の自動取得は不可能(https://github.com/typst/typst/
  discussions/2295)
  // 生成元のTypstファイル名は

}
} else {
  body
}
}

```

Listing 2: ブログ用に HTML を整形する Typst 関数

⁵context 周りは非常にややこしいので注意。トラップがいっぱいある。(今回のトラップは、context を関数の冒頭と、show の後の二回つけてしまうと、エラーがでないのに、出力がおかしくなってしまうことであろう。show の後の context は不要である。)